

Day-21-

- * A module is the basic building block in Verilog.
- * A module can be an element or a collection of lower-level design blocks.

Modelling styles:-

1. Gate-level Modelling / Structural
2. Switch level Modelling / Hierarchical
3. Behavioural modelling / Procedural modelling
4. Data flow modelling / continuous Assignment.

Modeling is described how a digital circuit works using Verilog code.

Data flow :-

- * It describes a circuit by specifying how data flows from inputs to outputs using Boolean expressions.
- * keyword used is assign. It is also called as continuous assignment modeling.

Syntax:- assign $y = a \& b$; assign output = expression;

- * It is used in combinational circuits, Boolean eqⁿ.
Decoders, Encoders, Multiplexers, Simple combinational RT.
- * we can have multiple assign statements (multiple I/p & O/p)

Behavioral:-

- * It describes what the circuit should do using procedural statements.

→ The major keyword = always, initial

- * Instead of directly writing the Boolean equation, we describe the behavior.

Syntax:- always (timing control) procedural - Statement.

Syntax for initial:- initial (timing control) procedural Statement

```

fx: always (negedge clk)
    begin
        { procedural statement
    end.
  
```

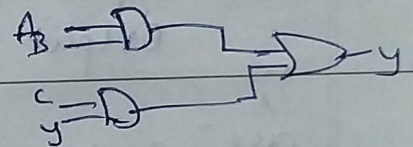
⇒ The signal assigned inside an always block should be a procedural variable that's what we make use of reg in behavioral.

behavioral can be represented as Sequential logic
+
Combinational logic

↳ Structural / Hierarchical :-

Structural modeling says about the components present and how they are connected

* It is useful when building large designs from smaller modules.



Hierarchical Design

* we can make use of existing (or) created module to create another module.

* Suppose if we created and 2 or before this can be used to create another module.

Gate-level modeling :-

Gate-level describes a circuit using built-in Verilog gate primitives.

Ex: and, or, not, nand, nor, xor, xnor, buff.

* It mainly focus of Gate primitives

* It says useful for understanding how gates are connected

Syntax:-

gate_type instance_name (o/p, input 1, input 2, ...);

Programs of all the modelling techniques using nand gate

(1) Dataflow → Nand

```
module nand_dataflow(
    input a, b,
    output y
);
    assign y = ~(a & b);
endmodule.
```

(3) Structural :-

```
module nand_structural(
    input a, b,
    output y
);
    nand_gate g1(
        .a(a),
        .b(b),
        .y(y)
    );
endmodule
```

(2) Behavioral

```
module nand_Behavioral(
    input a, b,
    output y
);
    always @(a, b) begin
        y = ~(a & b);
    end
endmodule..
```

(4) Gate-level:-

```
module nand_gate(y, a, b);
    input a, b;
    output y;
    nand g1(y, a, b);
endmodule
```

") $\overline{A \cdot B} = \overline{A} + \overline{B}$

* Readability :-

It means how easily a designer can understand the code, its purpose & the hardware.

⇒ It is easy to understand for small circuits.

⇒ A large design may contain hundreds or thousands of gate instances.

Readability of dataflow :-

⇒ Very concise, Boolean eq'ns are easy to recognize.

⇒ Good for simple combinational logic.

Readability of Behavioral :-

⇒ usually easiest for complex logic, if, case, loops etc.. make the designer's intention clear. Particularly useful for FSM's, control logic & complex RTL. Requires understanding of procedural Verilog & synthesis rules.

Readability of structural :-

⇒ Very good for understanding the architecture / block diagram.

⇒ Makes hierarchy and module reuse clear.

⇒ Large numbers of low-level connections can make the code difficult to follow.

Synthesizability :-

Synthesizability means whether a synthesis tool can convert the verilog description into actual h/w.

⇒ Not every legal verilog construct is synthesizable.

Gate level primitives are and, or, not, nand, nor. They are synthesizable.

Dataflow :- Continuous assignments are highly synthesizable.

Behavioral :- can be synthesizable, but not every behavioral construct is synthesizable.